



Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 7

Professional-Style Design Reviews



Ursinus
COLLEGE

Team Standup Meetings



Last Week

Detailed Design & Design Principles



Quick Review of Design Principles



Design Principle 1: Single Responsibility Principle (SRP)

A module should have one reason to change.

Not:

“It does one thing” ❌

But:

“It changes for one reason” ✅

A module should be responsible to one actor or concern in the system.

The goal is to reduce ripple effects when requirements change.



Design Principle 2: High Cohesion

Cohesion = How strongly related the responsibilities are within a module.

High Cohesion:

- Focused ✓
- Understandable ✓
- Maintainable ✓

Low Cohesion

- Random Functionality ✗
- Mixed Concerns ✗



Design Principle 3: Low Coupling

Coupling = How much one module depends on another.

Low Coupling:

- Change in A does not break B ✓
- Clear boundaries ✓
- Replaceable components ✓

High Coupling:

- Hidden dependencies ✗
- Ripple effects ✗
- Fragile systems ✗



Design Principle 4: Abstraction

Expose *what*

Hide *how*

Abstraction creates:

- Flexibility
- Testability
- Replaceability



Design Principle 5: Separation of Concerns

Different concerns should be handled by different modules.

Examples of concerns:

- Business logic
- Data access
- UI
- Validation
- Logging
- Security



Design Principle 6: Design for Testability

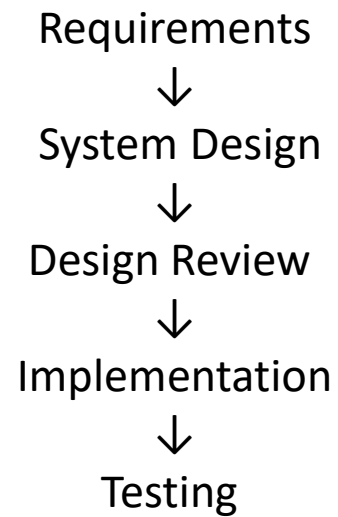
A well-designed module:

- Has clear inputs
- Has predictable outputs
- Has explicit preconditions
- Has measurable postconditions



Where Design Reviews Fit in Development

Typical development lifecycle:





What Makes Design Reviews Valuable

Design reviews help teams:

- Share architectural knowledge
- Catch misunderstandings
- Improve maintainability
- Ensure design consistency across the system



The Reviewer's Mindset

A design reviewer is not trying to:

- ✗ Prove the designer wrong
- ✗ Redesign the entire system

A design reviewer is trying to:

- ✓ Understand the design
- ✓ Identify risks
- ✓ Ask useful questions



Professional Design Feedback

Poor feedback:

- ✗ "This design is bad."
- ✗ "I don't like this class."

Good feedback:

- ✓ "This class appears to have multiple responsibilities."
- ✓ "How would we support additional payment providers?"



Questions Good Reviewers Ask

Examples of strong design review questions:

Responsibility

- What responsibility does this class have?

Coupling

- How dependent are these components on each other?

Extensibility

- What happens if we add new payment providers?

Failure modes

- What happens if this service fails?



Exercise 1: Design Review

Today we will simulate a real design review.

You will:

1. Read a short design proposal
2. Examine a UML diagram
3. Act as reviewers for the design

Your job is to identify:

- Risks
- Unclear responsibilities
- Potential improvements



Online Event Ticketing Platform: System Overview

Users can:

- Create accounts and log in
- Browse events
- Purchase tickets
- Receive tickets via email
- View their purchased tickets

Scale expectations:

- Several thousand users
- Hundreds of events



Online Event Ticket Platform: Functional Requirements

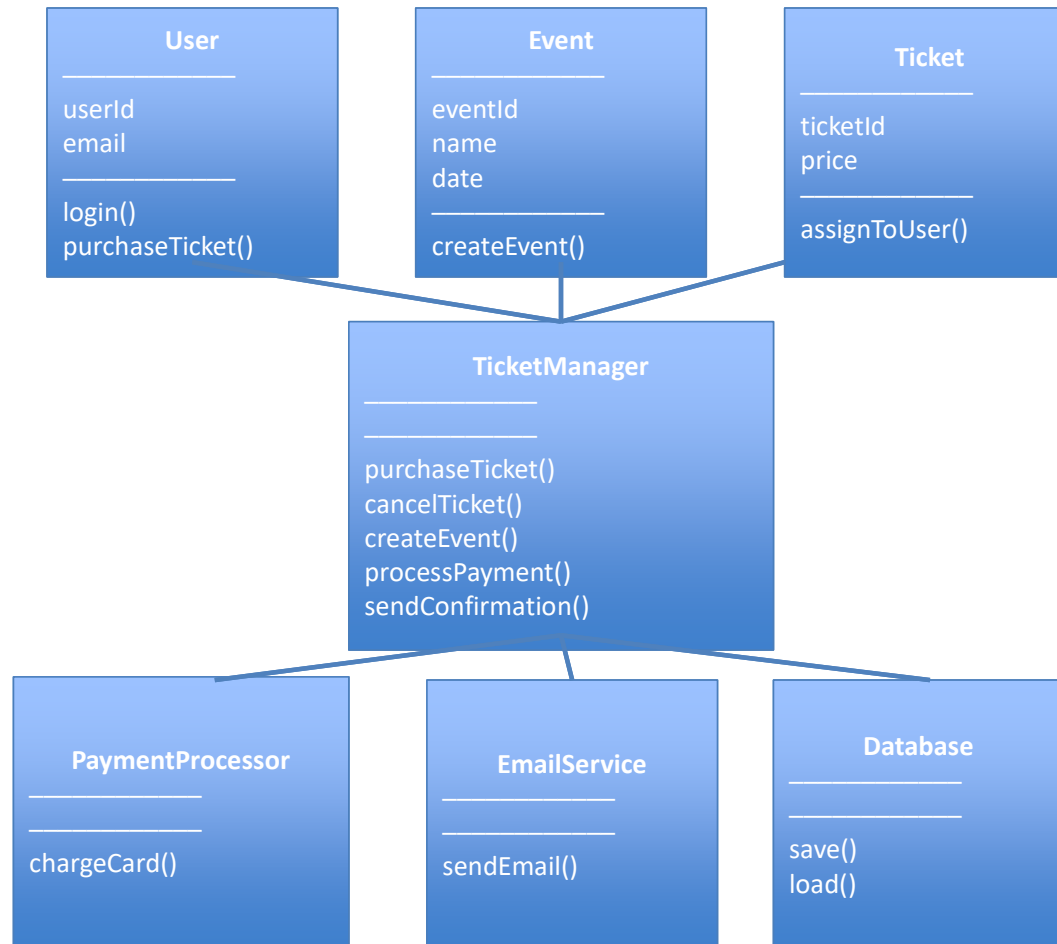
1. Users can create accounts and log in.
2. Users can browse available events.
3. Users can purchase tickets for an event.
4. The system processes payments.
5. Purchased tickets are emailed to the user.
6. Users can view previously purchased tickets.
7. The system records ticket purchases.



Design Review Discussion

- A junior developer proposed this design. What concerns do you see?
- Which class appears to have the most responsibilities?
- Which classes are tightly coupled?
- If we added refunds or ticket transfers, which classes would need to change?
- What changes might improve maintainability or extensibility?

Initial Design Proposal





Understanding the System

Key classes:

- User
- Event
- Ticket
- TicketManager
- PaymentProcessor
- EmailService
- Database



Design Review Activity

In groups of 3-4, identify:

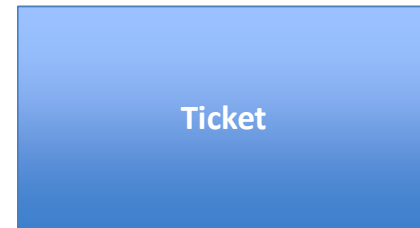
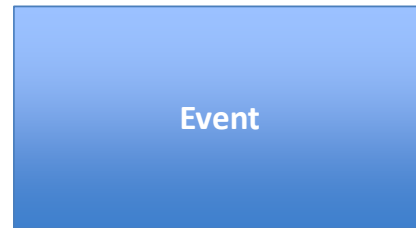
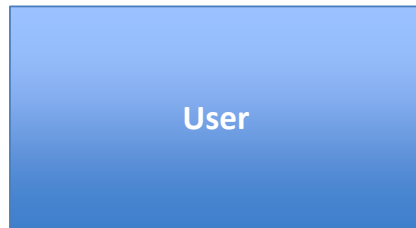
- 3 questions about the design
- 2 potential problems
- 1 improvement suggestion



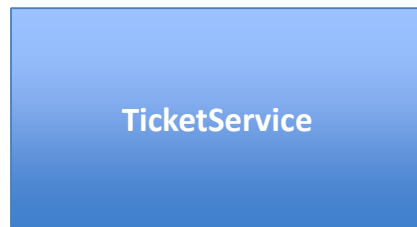
Exercise 2: Improving the Design

One Possible Refactoring

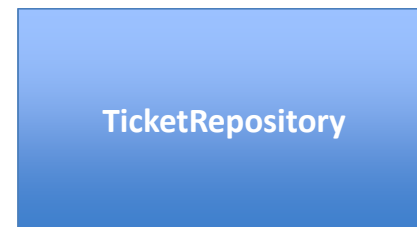
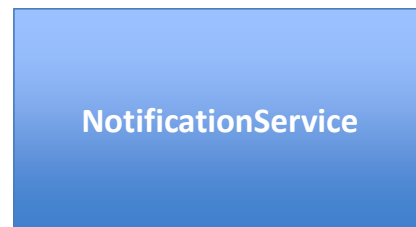
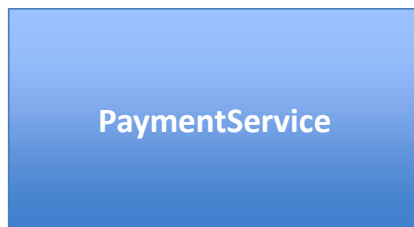
Domain Objects



Application Services



Supporting Services





2 Weeks From Now:

Implementation & Professional Coding Practices



Assignment: Weekly Standup Reflection

<https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection>

(Individual Assignment)

Due Feb 26 (and every week thereafter)



Assignment: Detailed Design Report

<https://rrostock1.github.io/Ursinus-CS375/Project/Design>

(Group Assignment)

Due March 26



Assignment: The Overdose

<https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose>

(Individual Assignment)

Due Apr 23



Due In 2 Weeks (March 26, by Midnight)

- Weekly Standup Reflection
- Detailed Design Report

